FACULTE DES SCIENCES BEN M'SICK
UNIVERSITÉ HASSAN II DE CASABLANCA

Master Intelligence Artificielle

Module : DevOps

RAPPORT DE PROJET

Mise en place d'une Infrastructure DevOps avec CI/CD, Kubernetes et Monitoring

Encadré par :

Pr. F. Benabbou

Réalisé par :

Salama BOUSSAID

Malika CHAABAN

Année universitaire : 2025 — 2026

Dépôt GitHub : https://github.com/malikachaaban/devops-master

Résumé :

Ce rapport présente la mise en place d'une infrastructure DevOps complète dans le cadre du module DevOps du Master DSBD & IA, encadré par Pr. F. Benabbou. L'objectif principal était de déployer une application conteneurisée de bout en bout sur une infrastructure cloud Microsoft Azure, en utilisant les outils et pratiques professionnels de l'industrie.

Le projet couvre l'ensemble du cycle de vie DevOps : provisionnement de l'infrastructure avec Terraform, configuration des serveurs avec Ansible, conteneurisation de l'application avec Docker, orchestration avec Kubernetes, et automatisation du déploiement via un pipeline CI/CD GitHub Actions.

Les résultats obtenus démontrent un déploiement automatisé complet : sur le cluster Kubernetes, incluant la construction et la publication automatique de l'image Docker sur DockerHub.

Table des Matières

1. Introduction 7
1.1. Contexte et Objectifs 7
1.2. Technologies Utilisées 7
1.3. Architecture du Projet 7
2. Configuration de l'Infrastructure Cloud 9
2.1. Compte Microsoft Azure Étudiant 9
2.2. Installation des Outils 10
2.3. Fichiers de Configuration Terraform 11
2.4. Exécution de Terraform 13
3. Création et Configuration des Machines Virtuelles 15
3.1. Déploiement des VMs 15
3.2. Vérification et Réseau 16
4. Installation de Docker et Kubernetes 19
4.1. Installation de Docker 19
4.2. Préparation du Système 19
4.3. Installation de Kubernetes 20
4.4. Initialisation du Cluster 21
4.5. Rattachement du Worker 21
5. Développement et Conteneurisation de l'Application 22
5.1. Description de l'Application 22
5.2. Construction et Publication Docker 24
6. Déploiement sur Kubernetes 25
6.1. Manifests Kubernetes 25
6.2. Application et Vérification 26
7. Pipeline CI/CD avec GitHub Actions 27
7.1. Structure du Dépôt GitHub 27
7.2. Configuration du Pipeline 27
7.3. Secrets et Résultats 29
8. Automatisation avec Ansible 31
8.1. Inventaire 31
8.2. Playbook Docker (install_docker.yml) 31
8.3. Playbook Kubernetes (install_k8s.yml) 31
8.4. Reproduction de l'Infrastructure 32

9. Monitoring avec Grafana 33
9.1. Installation de Grafana avec Helm 33
9.2. Accès au Dashboard Grafana 34
9.3. Vérification du Déploiement 34
10. Conclusion 35
10.1. Récapitulatif 35
10.2. Compétences Acquises 36
11. Références 37

Liste des Figures

Figure 1: Architecture du projet....8
Figure 2: Page d'accueil Microsoft Azure — Présentation des offres étudiantes disponibles....9
Figure 3: Dashboard Azure Education — 100 dollars de crédits disponibles, expiration 14/03/2027....9
Figure 4: Installation de Microsoft Azure CLI 32-bit et acceptation des termes de licence....10
Figure 5: Vérification Azure CLI v2.84.0 et Terraform v1.14.7....10
Figure 6: Création de provider.tf et variables.tf — Configuration du fournisseur Azure....11
Figure 7: Création de outputs.tf et terraform.tfvars — Définition des sorties et valeurs....11
Figure 8: Fichier main.tf — Définition des ressources Azure pour k8s-master et k8s-worker....12
Figure 9: Terraform init — Téléchargement des providers hashicorp/azurerm v3.117.1, tls v4.2.1....13
Figure 10: Terraform validate — Message « Success! The configuration is valid. »....13
Figure 11: Terraform plan — Plan d'exécution des 14 ressources incluant les 2 VMs Azure....14
Figure 12: Déploiement Azure terminé — VM k8s-master créée avec succès le 14/03/2026....15
Figure 13: Déploiement k8s-worker — Ressources réseau (vnet, NSG, IP publique) créées....15
Figure 14: Azure Virtual Machines — k8s-master (Spain Central) et k8s-worker (Norway East) Running....16
Figure 15: Règle NSG devops-all-ports — Ports 22, 80, 443, 6443, 8080, 10250, 30080 autorisés....16
Figure 16: : Paramètres réseau k8s-master — IP publique 68.221.161.55, IP privée 10.0.0.4....17
Figure 17: Connexion SSH réussie vers k8s-master — Terminal Ubuntu 24.04 LTS accessible....18
Figure 18: Connexion SSH réussie vers k8s-worker — Terminal Ubuntu 24.04 LTS accessible....18
Figure 19: Vérification initiale — Docker et Kubernetes absents, démarrage de l'installation....19
Figure 20: Configuration kernel — sudo swapoff -a, modules overlay/br_netfilter, paramètres sysctl....20
Figure 21: : Installation kubeadm, kubelet, kubectl v1.28 — Paquets mis en attente avec hold....20
Figure 22: : kubeadm init réussi — Génération certificats, Flannel installé, master en état Ready....21

Figure 23: kubectl get nodes — k8s-master (control-plane) et k8s-worker Ready, version v1.28 22
Figure 24: Interface de l'application 23
Figure 25: Code source de l'application 23
Figure 26: : docker build réussi — Construction en 12s, push vers DockerHub salama12/devops-app:1.0 24
Figure 27: DockerHub — Dépôt salama12/devops-app visible publiquement, image poussée 25
Figure 28: Fichier deployment.yml 25
Figure 29: Le fichier service.yml 26
Figure 30: : kubectl get pods -n devops-app — 2 pods Running, service NodePort 80:30080/TCP créé 26
Figure 31: curl /health depuis worker — Réponse {"status":"ok"} confirmant le déploiement 26
Figure 32: : Dépôt GitHub devops-master — Structure complète du projet, 9 commits 27
Figure 33: Fichier de tests unitaires test_app.py — fixtures de réinitialisation et configuration de l'application Flask en mode test 28
Figure 34: Initialisation du dépôt et push du projet vers GitHub (branche main) 29
Figure 35: Configuration du pipeline CI/CD GitHub Actions pour le build, les tests et le déploiement automatique 29
Figure 36: Secrets GitHub configurés — 4 secrets chiffrés pour sécuriser le déploiement automatique 30
Figure 37: GitHub Actions — Pipeline CI/CD #4 vert en 27 secondes, toutes les étapes réussies 30
Figure 38: Le fichier inventory.ini des groupes hôtes 31
Figure 39: Le fichier install_docker.yml 31
Figure 40: Le fichier install_k8s.yml 32
Figure 41: Etapes d'installation 33
Figure 42: Exposition du service Grafana via NodePort sur Kubernetes 34
Figure 43: Vérification de l'état du pod Grafana dans le namespace monitoring . 34
Figure 44: Affichage détaillé du pod Grafana avec son adresse IP et son nœud d'exécution 34
Figure 45: Structure du projet. 35

1. Introduction

1.1. Contexte et Objectifs

Le DevOps est une approche culturelle et technique qui vise à unifier le développement logiciel et les opérations informatiques. Dans le cadre du module DevOps du Master DSBD & IA de la Faculté des Sciences Ben M'Sick, ce projet a pour objectif de mettre en pratique les principes fondamentaux du DevOps en réalisant une infrastructure complète de bout en bout.

L'objectif est de créer une infrastructure DevOps complète sur Microsoft Azure, permettant de déployer automatiquement une application conteneurisée à l'aide d'une chaîne CI/CD entièrement automatisée, simulant les conditions réelles d'une infrastructure de production.

1.2. Technologies Utilisées

Outil	Rôle dans le projet	Version
Microsoft Azure	Fournisseur cloud pour l' hébergement VMs	Azure for Students
Terraform	Infrastructure as Code pour la création des VMs	v1.14.7
Azure CLI	Interface ligne de commande Azure	v2.84.0
Ansible	Configuration automatique des serveurs	Playbooks YAML
Docker	Conteneurisation de l'application	v29.3.0
Kubernetes / k3s	Orchestration des conteneurs	v1.34.5+k3s1
Python Flask	Développement de l'API REST	3.11-slim
GitHub Actions	Pipeline CI/CD automatisé	YAML workflow
DockerHub	Registre d'images Docker	Public repository
Grafana	Dashboard de surveillance (monitoring)	

1.3. Architecture du Projet

L'architecture repose sur deux machines virtuelles Azure formant un cluster Kubernetes. Le nœud master (IP : 68.221.161.55) gère le plan de contrôle, tandis que le worker (IP : 20.251.155.11) exécute les pods. Le déploiement est automatisé via GitHub Actions déclenché à chaque push sur la branche principale :

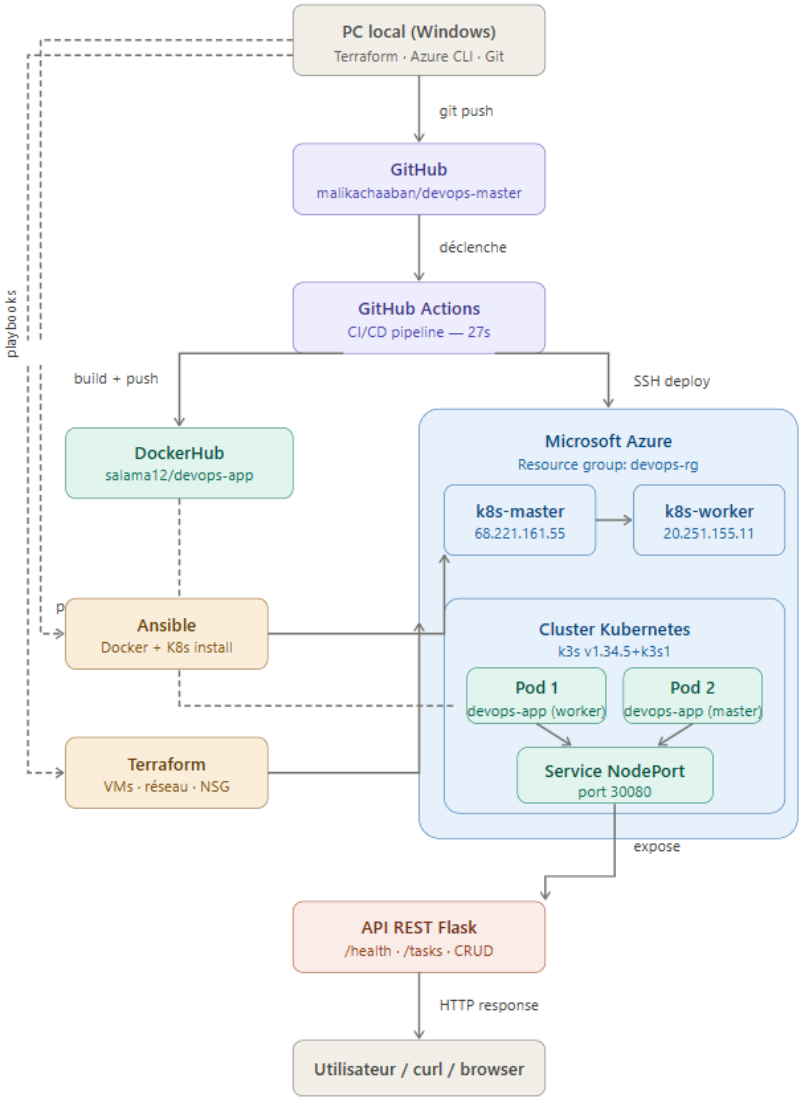


Figure 1: Architecture du projet

2. Configuration de l'Infrastructure Cloud

2.1. Compte Microsoft Azure Étudiant

La première étape a consisté à créer un compte Microsoft Azure for Students. Ce compte offre 100 dollars de crédits utilisables sur les services Azure, valables 365 jours, sans carte bancaire.

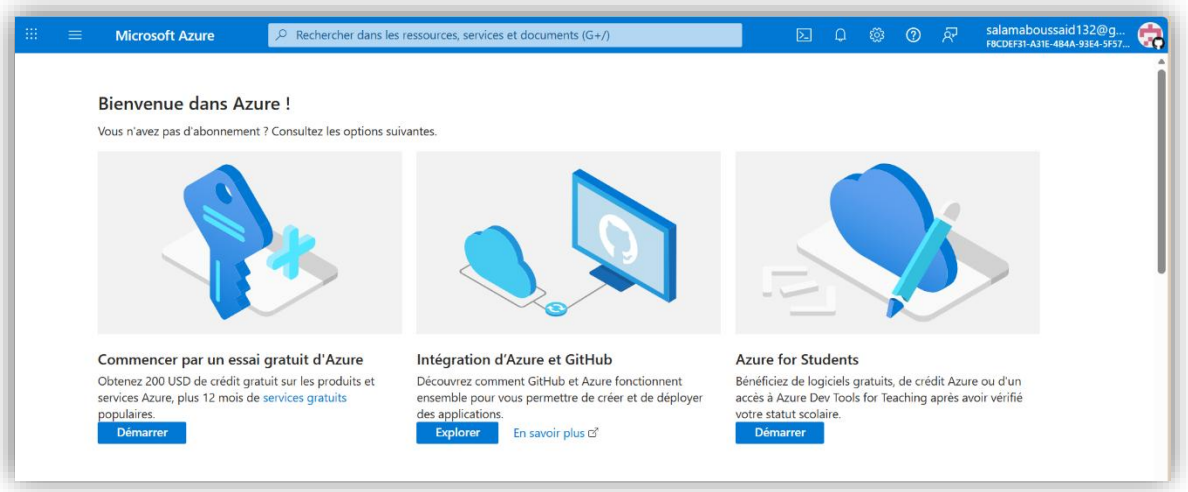


Figure 2: Page d'accueil Microsoft Azure — Présentation des offres étudiantes disponibles

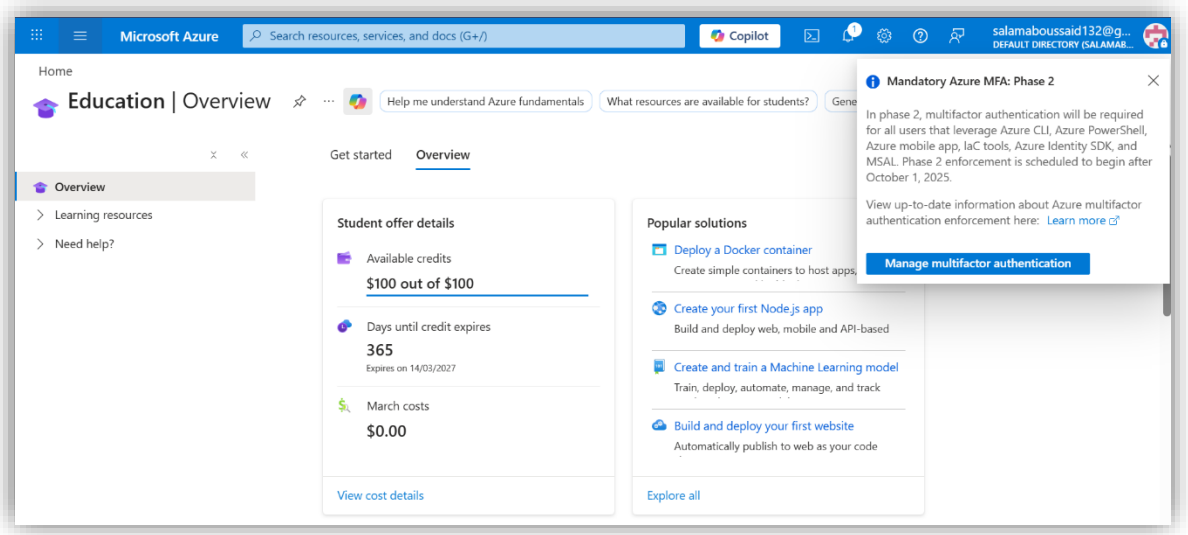


Figure 3: Dashboard Azure Education — 100 dollars de crédits disponibles, expiration 14/03/2027

2.2. Installation des Outils

Azure CLI (version 2.84.0) et Terraform (v1.14.7) ont été installés sur le poste Windows 11. La commande az account show a permis de récupérer l'identifiant de souscription Azure nécessaire pour Terraform.

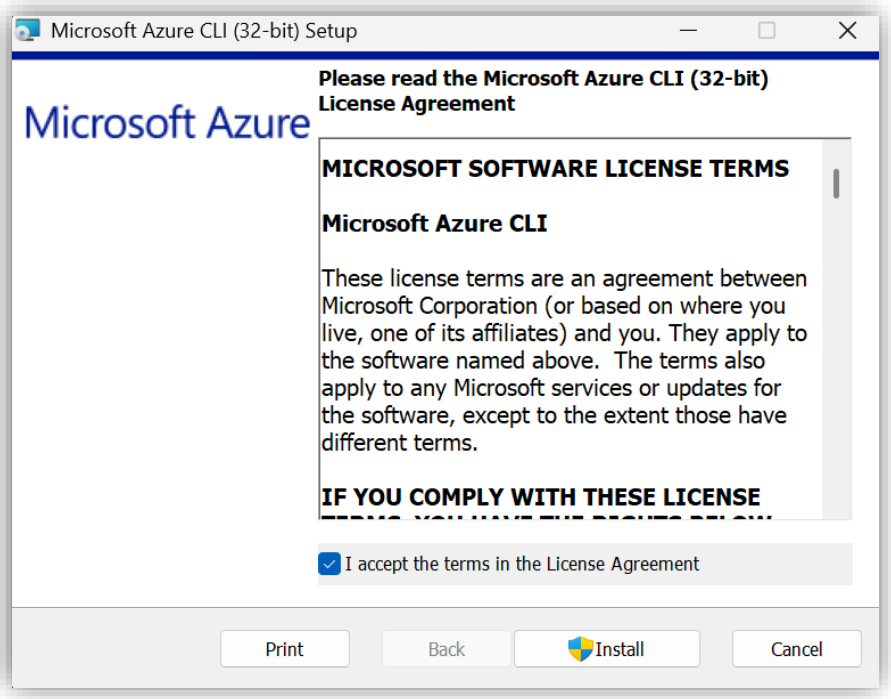


Figure 4: Installation de Microsoft Azure CLI 32-bit et acceptation des termes de licence

```
PS C:\WINDOWS\system32> az --version
azure-cli                         2.84.0

core                              2.84.0
telemetry                          1.1.0

Dependencies:
msal                            1.35.0b1
azure-mgmt-resource               24.0.0

Python location 'C:\Program Files (x86)\Microsoft SDKs\Azure\CLI2\python.exe'
Config directory 'C:\Users\Salama\.azure'
Extensions directory 'C:\Users\Salama\.azure\cliextensions'

Python (Windows) 3.13.11 (tags/v3.13.11:6278944, Dec  5 2025, 16:17:02) [MSC v.1944 32 bit (Intel)]

Legal docs and information: aka.ms/AzureCliLegal


Your CLI is up-to-date.
PS C:\WINDOWS\system32> $env:PATH += ";C:\terraform"
PS C:\WINDOWS\system32> terraform --version
Terraform v1.14.7
on windows_amd64
```

Figure 5: Vérification Azure CLI v2.84.0 et Terraform v1.14.7

2.3. Fichiers de Configuration Terraform

Quatre fichiers HCL ont été créés dans le dossier terraform/ : provider.tf (définit le fournisseur azurerm), variables.tf (déclare les variables), terraform.tfvars (contient les valeurs concrètes) et outputs.tf (définit les sorties après déploiement).

```
PS C:\Users\Salama\Desktop\devops-project\terraform> @"
>> terraform {
>>   required_providers {
>>     azurerm = {
>>       source  = "hashicorp/azurerm"
>>       version = "~> 3.0"
>>     }
>>   }
>> }
>>
>> provider "azurerm" {
>>   features {}
>>   subscription_id = var.subscription_id
>> }
>> "@ | Out-File -FilePath provider.tf -Encoding utf8
```

```
PS C:\Users\Salama\Desktop\devops-project\terraform> @"
>> variable "subscription_id" {}
>> variable "location" { default = "eastus" }
>> variable "admin_username" { default = "azureuser" }
>> "@ | Out-File -FilePath variables.tf -Encoding utf8
PS C:\Users\Salama\Desktop\devops-project\terraform>
```

Figure 6: Création de provider.tf et variables.tf — Configuration du fournisseur Azure

```
PS C:\Users\Salama\Desktop\devops-project\terraform> az account show --query id -o tsv
9c83ee95-f8c0-476b-a2ef-cc036c67c101
PS C:\Users\Salama\Desktop\devops-project\terraform> @"
>>
>> subscription_id = "9c83ee95-f8c0-476b-a2ef-cc036c67c101"
>>
>> "@ | Out-File -FilePath terraform.tfvars -Encoding utf8
PS C:\Users\Salama\Desktop\devops-project\terraform>
```

```
PS C:\Users\Salama\Desktop\devops-project\terraform> @"
>> output "master_ip" { value = azurerm_public_ip.master.ip_address }
>> output "worker_ip"  { value = azurerm_public_ip.worker.ip_address }
>> output "ssh_key_path" { value = "`${path.module}/devops-key.pem" }
>> "@ | Out-File -FilePath outputs.tf -Encoding utf8
```

Figure 7: Création de outputs.tf et terraform.tfvars — Définition des sorties et valeurs

Le fichier main.tf définit toutes les ressources Azure : groupe de ressources devops-rg, réseau virtuel 10.0.0.0/16, sous-réseau 10.0.1.0/24, NSG, interfaces réseau et deux VMs Ubuntu 22.04 Standard_B2s.

```
PS C:\Users\Salama\Desktop\devops-project\terraform> @"
>> resource "azurerm_resource_group" "devops" {
>>   name     = "devops-rg"
>>   location = var.location
>> }
>>
>> resource "azurerm_virtual_network" "devops" {
>>   name                = "devops-vnet"
>>   address_space       = ["10.0.0.0/16"]
>>   location            = var.location
>>   resource_group_name = azurerm_resource_group.devops.name
>> }
>>
>> resource "azurerm_subnet" "devops" {
>>   name                 = "devops-subnet"
>>   resource_group_name  = azurerm_resource_group.devops.name
>>   virtual_network_name = azurerm_virtual_network.devops.name
>>   address_prefixes     = ["10.0.1.0/24"]
>> }
>>
>> resource "azurerm_network_security_group" "devops" {
```

```
>>   }
>> }
>>
>> resource "azurerm_linux_virtual_machine" "worker" {
>>   name                = "k8s-worker"
>>   location            = var.location
>>   resource_group_name = azurerm_resource_group.devops.name
>>   size                = "Standard_B2s"
>>   admin_username      = var.admin_username
>>   network_interface_ids = [azurerm_network_interface.worker.id]
>>
>>   admin_ssh_key {
>>     username   = var.admin_username
>>     public_key = tls_private_key.ssh.public_key_openssh
>>   }
>>
>>   os_disk {
>>     caching              = "ReadWrite"
>>     storage_account_type = "Standard_LRS"
>>   }
>>
>>   source_image_reference {
>>     publisher = "Canonical"
>>     offer     = "0001-com-ubuntu-server-jammy"
>>     sku       = "22_04-lts"
>>     version   = "latest"
>>   }
>> }
>> "@ | Out-File -FilePath main.tf -Encoding utf8
```

```
PS C:\Users\Salama\Desktop\devops-project\terraform> ls


    Répertoire : C:\Users\Salama\Desktop\devops-project\terraform


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        14/03/2026     18:01           5971 main.tf
-a----        14/03/2026     17:58            205 outputs.tf
-a----        14/03/2026     17:54            206 provider.tf
-a----        14/03/2026     17:57             63 terraform.tfvars
-a----        14/03/2026     17:56            134 variables.tf
```

Figure 8: Fichier main.tf — Définition des ressources Azure pour k8s-master et k8s-worker

2.4. Exécution de Terraform

Le workflow Terraform suit trois étapes : terraform init (télécharge les providers), terraform validate (vérifie la syntaxe) et terraform plan (affiche le plan d'exécution). Les 14 ressources à créer ont été confirmées.

```
PS C:\Users\Salama\Desktop\devops-project\terraform> terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/azurerm versions matching "~> 3.0"...
- Finding latest version of hashicorp/tls...
- Finding latest version of hashicorp/local...
- Installing hashicorp/azurerm v3.117.1...
- Installed hashicorp/azurerm v3.117.1 (signed by HashiCorp)
- Installing hashicorp/tls v4.2.1...
- Installed hashicorp/tls v4.2.1 (signed by HashiCorp)
- Installing hashicorp/local v2.7.0...
- Installed hashicorp/local v2.7.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Figure 9: Terraform init — Téléchargement des providers hashicorp/azurerm v3.117.1, tls v4.2.1

```
PS C:\Users\Salama\Desktop\devops-project\terraform> terraform validate
Success! The configuration is valid.
```

Figure 10: Terraform validate — Message « Success! The configuration is valid. »

```
PS C:\Users\Salama\Desktop\devops-project\terraform> terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
  + create

Terraform will perform the following actions:

  # azurerm_linux_virtual_machine.master will be created
  + resource "azurerm_linux_virtual_machine" "master" {
      + admin_username                                       = "azureuser"
      + allow_extension_operations                           = true
      + bypass_platform_safety_checks_on_user_schedule_enabled = false
      + computer_name                                        = (known after apply)
      + disable_password_authentication                      = true
      + disk_controller_type                                 = (known after apply)
      + extensions_time_budget                               = "PT1H30M"
      + id                                                   = (known after apply)
      + location                                             = "westeurope"
      + max_bid_price                                        = -1
      + name                                                 = "k8s-master"
      + network_interface_ids                                = (known after apply)
      + patch_assessment_mode                                = "ImageDefault"
      + patch_mode                                           = "ImageDefault"
      + platform_fault_domain                                = -1
      + priority                                             = "Regular"
      + private_ip_address                                   = (known after apply)
      + private_ip_addresses                                 = (known after apply)
      + provision_vm_agent                                   = true
      + id                    = (known after apply)
    }

  # tls_private_key.ssh will be created
  + resource "tls_private_key" "ssh" {
      + algorithm                     = "RSA"
      + ecdsa_curve                   = "P224"
      + id                            = (known after apply)
      + private_key_openssh           = (sensitive value)
      + private_key_pem               = (sensitive value)
      + private_key_pem_pkcs8         = (sensitive value)
      + public_key_fingerprint_md5    = (known after apply)
      + public_key_fingerprint_sha256 = (known after apply)
      + public_key_openssh            = (known after apply)
      + public_key_pem                = (known after apply)
      + rsa_bits                      = 4096
    }

Plan: 14 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + master_ip    = (known after apply)
  + ssh_key_path = "./devops-key.pem"
  + worker_ip    = (known after apply)


Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if
you run "terraform apply" now.
PS C:\Users\Salama\Desktop\devops-project\terraform>
```

Figure 11: Terraform plan — Plan d'exécution des 14 ressources incluant les 2 VMs Azure

3. Création et Configuration des Machines Virtuelles

3.1. Déploiement des VMs

Les VMs ont été créées sur Azure avec Ubuntu Server 22.04 LTS x64 Gen2. Le master a été déployé en Spain Central et le worker en Norway East. La taille Standard_B2s (2 vCPUs, 8 Go RAM) a été choisie pour un bon rapport performances/coût dans le cadre du tier gratuit.

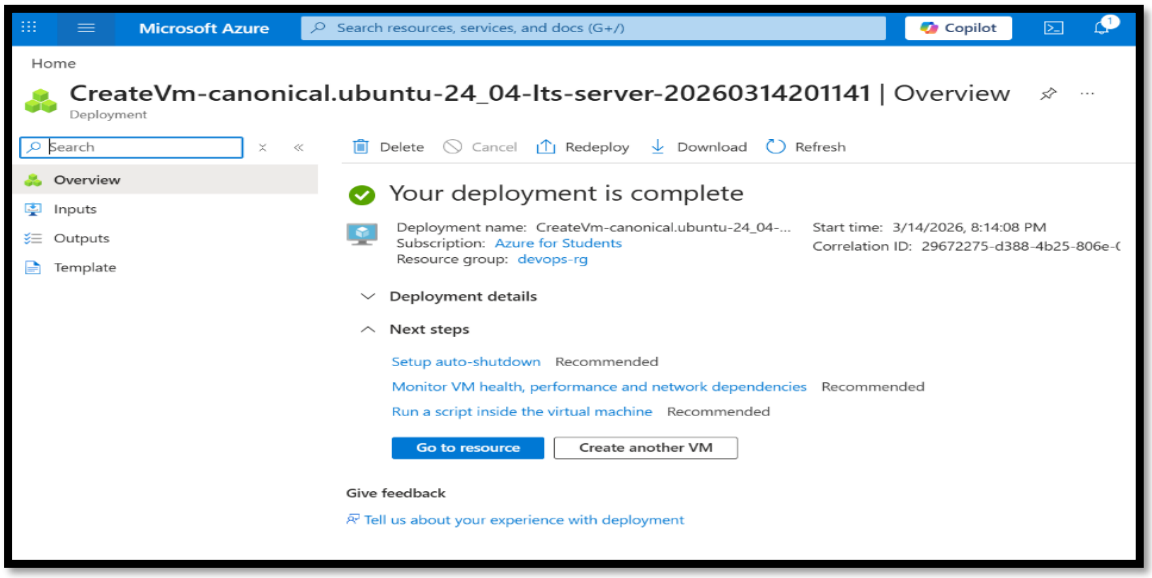


Figure 12: Déploiement Azure terminé — VM k8s-master créée avec succès le 14/03/2026

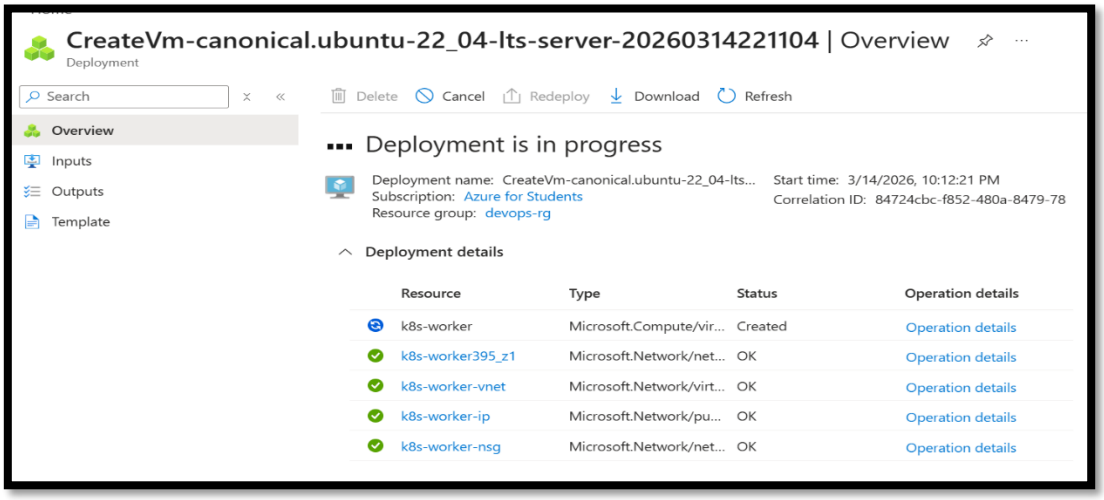


Figure 13: Déploiement k8s-worker — Ressources réseau (vnet, NSG, IP publique) créées

3.2. Vérification et Réseau

Après déploiement, les deux VMs apparaissent en état Running dans le portail Azure. Une règle NSG (devops-all-ports, priorité 100) autorise les ports nécessaires : 22 (SSH), 443 (HTTPS), 6443 (API Kubernetes), 10250 (Kubelet), et 30080 (service applicatif).

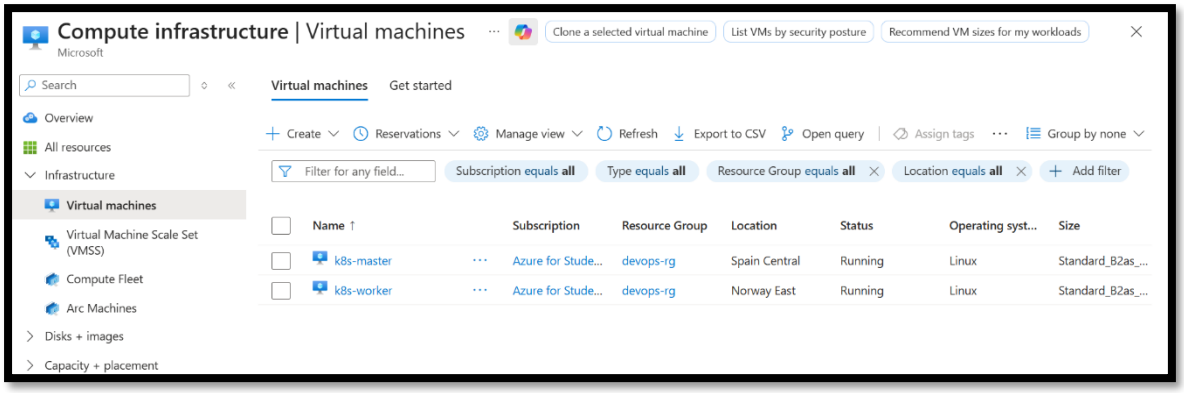


Figure 14: Azure Virtual Machines — k8s-master (Spain Central) et k8s-worker (Norway East) Running

Figure 15: Règle NSG devops-all-ports — Ports 22, 80, 443, 6443, 8080, 10250, 30080 autorisés

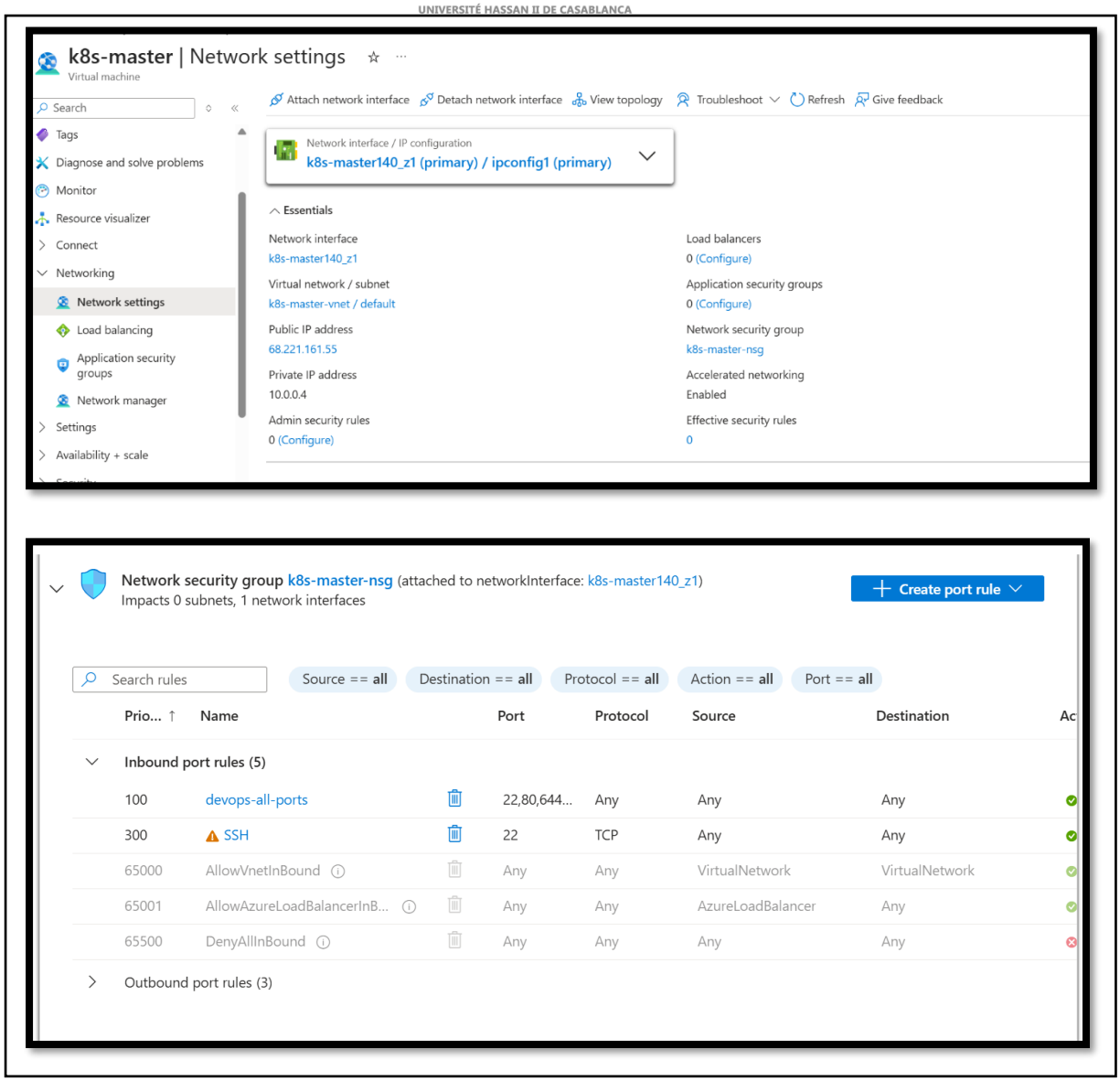


Figure 16: : Paramètres réseau k8s-master – IP publique 68.221.161.55, IP privée 10.0.0.4

```
PS C:\Users\Salama> az vm list-ip-addresses --resource-group devops-rg --output table
VirtualMachine    PublicIPAddresses    PrivateIPAddresses
----------------  -------------------  --------------------
k8s-worker        20.251.155.11        10.0.0.4
k8s-master        68.221.161.55        10.0.0.4
PS C:\Users\Salama> ssh -i C:\Users\Salama\.ssh\devops-key.pem azureuser@68.221.161.55
The authenticity of host '68.221.161.55 (68.221.161.55)' can't be established.
ED25519 key fingerprint is SHA256:P7FtnPKxWyyxOrH0xGOcf7JLXs8K3bnIVqbYsUX+imE.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '68.221.161.55' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1008-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Sat Mar 14 22:51:37 UTC 2026

  System load:  0.0               Processes:             131
  Usage of /:   5.8% of 28.02GB   Users logged in:       0
  Memory usage: 4%                IPv4 address for eth0: 10.0.0.4

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

azureuser@k8s-master:~$
```

Figure 17: Connexion SSH réussie vers k8s-master — Terminal Ubuntu 24.04 LTS accessible

```
Expanded Security Maintenance for Applications is not enabled.

15 updates can be applied immediately.
6 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.


*** System restart required ***
Last login: Wed Mar 18 23:46:21 2026 from 102.52.8.189
azureuser@k8s-worker:~$
```

Figure 18: Connexion SSH réussie vers k8s-worker — Terminal Ubuntu 24.04 LTS accessible

4. Installation de Docker et Kubernetes

4.1. Installation de Docker

L'installation de Docker a suivi la procédure officielle Docker Inc. pour Ubuntu : ajout de la clé GPG officielle, ajout du dépôt APT, installation du paquet docker.io, activation du service via systemctl. Un problème de permissions sur le socket Docker a été résolu en ajoutant l'utilisateur au groupe docker.

```
azureuser@k8s-master:~$ docker --version
Command 'docker' not found, but can be installed with:
sudo apt install docker.io      # version 28.2.2-0ubuntu1~24.04.1, or
sudo apt install podman-docker  # version 4.9.3+ds1-1ubuntu0.2
azureuser@k8s-master:~$ kubectl version --client
Command 'kubectl' not found, did you mean:
  command 'kubectx' from deb kubectx (0.9.5-1ubuntu0.4)
Try: sudo apt install <deb name>
azureuser@k8s-master:~$ kubeadm version
kubeadm: command not found
azureuser@k8s-master:~$ kubelet --version
kubelet: command not found
azureuser@k8s-master:~$ sudo apt update && sudo apt upgrade -y
```

```
azureuser@k8s-master:~$ sudo install -m 0755 -d /etc/apt/keyrings
azureuser@k8s-master:~$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/key
rings/docker.gpg
azureuser@k8s-master:~$ sudo chmod a+r /etc/apt/keyrings/docker.gpg
azureuser@k8s-master:~$ echo "deb [arch=amd64 signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/li
nux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list
deb [arch=amd64 signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu noble stable
azureuser@k8s-master:~$ sudo apt update
```

```
azureuser@k8s-master:~$ sudo systemctl enable docker && sudo systemctl start docker
Synchronizing state of docker.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable docker
azureuser@k8s-master:~$
```

Figure 19: Vérification initiale — Docker et Kubernetes absents, démarrage de l'installation

4.2. Préparation du Système

Avant l'installation de Kubernetes, plusieurs configurations système sont obligatoires : désactivation du swap (Kubernetes ne fonctionne pas avec swap actif), chargement des modules kernel overlay et br_netfilter, activation du forwarding IPv4 et des iptables via les paramètres sysctl.

```
azureuser@k8s-master:~$ sudo swapoff -a
azureuser@k8s-master:~$ sudo sed -i '/swap/d' /etc/fstab
azureuser@k8s-master:~$
```

```
azureuser@k8s-master:~$ sudo modprobe overlay
azureuser@k8s-master:~$ sudo modprobe br_netfilter
azureuser@k8s-master:~$ cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
> net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
EOF
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
azureuser@k8s-master:~$ sudo sysctl --system
* Applying /usr/lib/sysctl.d/10-apparmor.conf ...
* Applying /etc/sysctl.d/10-bufferbloat.conf ...
* Applying /etc/sysctl.d/10-console-messages.conf ...
* Applying /etc/sysctl.d/10-ipv6-privacy.conf ...
* Applying /etc/sysctl.d/10-kernel-hardening.conf ...
* Applying /etc/sysctl.d/10-magic-sysrq.conf ...
* Applying /etc/sysctl.d/10-map-count.conf ...
* Applying /etc/sysctl.d/10-network-security.conf ...
* Applying /etc/sysctl.d/10-ptrace.conf ...
* Applying /etc/sysctl.d/10-zeropage.conf ...
* Applying /usr/lib/sysctl.d/50-pid-max.conf ...
```

Figure 20: Configuration kernel — sudo swapoff -a, modules overlay/br_netfilter, paramètres sysctl

4.3. Installation de Kubernetes

Les paquets kubelet, kubeadm et kubectl (version 1.28) ont été installés depuis le dépôt officiel kubernetes.io. La commande apt-mark hold bloque les mises à jour automatiques pour garantir la stabilité du cluster.

```
azureuser@k8s-master:~$ curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.28/deb/Release.key | sudo gpg --dearmor -o /
etc/apt/keyrings/kubernetes.gpg
azureuser@k8s-master:~$ echo "deb [signed-by=/etc/apt/keyrings/kubernetes.gpg] https://pkgs.k8s.io/core:/stable:/v1.
28/deb/ /" | sudo tee /etc/apt/sources.list.d/kubernetes.list
deb [signed-by=/etc/apt/keyrings/kubernetes.gpg] https://pkgs.k8s.io/core:/stable:/v1.28/deb/ /
azureuser@k8s-master:~$ sudo apt update
Hit:1 http://azure.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://azure.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://azure.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 http://azure.archive.ubuntu.com/ubuntu noble-security InRelease
Hit:6 https://download.docker.com/linux/ubuntu noble InRelease
Get:5 https://prod-cdn.packages.k8s.io/repositories/isv:/kubernetes:/core:/stable:/v1.28/deb  InRelease [1192 B]
Get:7 https://prod-cdn.packages.k8s.io/repositories/isv:/kubernetes:/core:/stable:/v1.28/deb  Packages [21.3 kB]
Fetched 22.5 kB in 1s (44.3 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
azureuser@k8s-master:~$ sudo apt install -y kubelet kubeadm kubectl
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  conntrack cri-tools kubernetes-cni
The following NEW packages will be installed:
```

```
azureuser@k8s-master:~$ docker --version
Docker version 29.3.0, build 5927d80
azureuser@k8s-master:~$ kubectl version --client
Client Version: v1.28.15
Kustomize Version: v5.0.4-0.20230601165947-6ce0bf390ce3
azureuser@k8s-master:~$ kubeadm version
kubeadm version: &version.Info{Major:"1", Minor:"28", GitVersion:"v1.28.15", GitCommit:"841856557ef0f6a399096c42635d
114d6f2cf7f4", GitTreeState:"clean", BuildDate:"2024-10-22T20:33:16Z", GoVersion:"go1.22.8", Compiler:"gc", Platform
:"linux/amd64"}
azureuser@k8s-master:~$
```

```
azureuser@k8s-master:~$ sudo apt-mark hold kubelet kubeadm kubectl
kubelet set on hold.
kubeadm set on hold.
kubectl set on hold.
azureuser@k8s-master:~$
```

Figure 21: : Installation kubeadm, kubelet, kubectl v1.28 — Paquets mis en attente avec hold

4.4. Initialisation du Cluster

La commande kubeadm init --pod-network-cidr=192.168.0.0/16 a initialisé le cluster sur le master, générant les certificats TLS pour tous les composants. Le fichier kubeconfig a été copié et Flannel installé comme plugin réseau pour la communication entre pods.

```
azureuser@k8s-master:~$ sudo kubeadm init --pod-network-cidr=192.168.0.0/16
I0314 23:18:02.178966    6138 version.go:256] remote version is much newer: v1.35.2; falling back to: stable-1.28
[init] Using Kubernetes version: v1.28.15
[preflight] Running pre-flight checks
[preflight] Pulling images required for setting up a Kubernetes cluster
[preflight] This might take a minute or two, depending on the speed of your internet connection
[preflight] You can also perform this action in beforehand using 'kubeadm config images pull'
[certs] Using certificateDir folder "/etc/kubernetes/pki"
[certs] Generating "ca" certificate and key
[certs] Generating "apiserver" certificate and key
[certs] apiserver serving cert is signed for DNS names [k8s-master kubernetes kubernetes.default kubernetes.default.svc ku
rnetes.default.svc.cluster.local] and IPs [10.96.0.1 10.0.0.4]
[certs] Generating "apiserver-kubelet-client" certificate and key
[certs] Generating "front-proxy-ca" certificate and key
[certs] Generating "front-proxy-client" certificate and key
[certs] Generating "etcd/ca" certificate and key
[certs] Generating "etcd/server" certificate and key
[certs] etcd/server serving cert is signed for DNS names [k8s-master localhost] and IPs [10.0.0.4 127.0.0.1 ::1]
[certs] Generating "etcd/peer" certificate and key
[certs] etcd/peer serving cert is signed for DNS names [k8s-master localhost] and IPs [10.0.0.4 127.0.0.1 ::1]
[certs] Generating "etcd/healthcheck-client" certificate and key
[certs] Generating "apiserver-etcd-client" certificate and key
[certs] Generating "sa" key and public key
```

```
azureuser@k8s-master:~$ mkdir -p $HOME/.kube
azureuser@k8s-master:~$ sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
azureuser@k8s-master:~$ sudo chown $(id -u):$(id -g) $HOME/.kube/config
azureuser@k8s-master:~$
```

```
azureuser@k8s-master:~$ kubectl apply -f https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml
namespace/kube-flannel created
serviceaccount/flannel created
clusterrole.rbac.authorization.k8s.io/flannel created
clusterrolebinding.rbac.authorization.k8s.io/flannel created
configmap/kube-flannel-cfg created
daemonset.apps/kube-flannel-ds created
azureuser@k8s-master:~$ kubectl get nodes
NAME          STATUS   ROLES           AGE     VERSION
k8s-master    Ready    control-plane   6m23s   v1.28.15
azureuser@k8s-master:~$
```

```
azureuser@k8s-master:~$ sudo cat /var/lib/rancher/k3s/server/node-token
K106653b38c906c3432f19f3af57489d2c7a15139f1da62a59bd8003264cb4a9d57::server:9e2f28b29a0dae2c060f4687b0a31cad
azureuser@k8s-master:~$
```

Figure 22: : kubeadm init réussi — Génération certificats, Flannel installé, master en état Ready

4.5. Rattachement du Worker

Le cluster Kubernetes a été entièrement déployé en utilisant kubeadm, permettant une configuration standard et conforme aux pratiques professionnelles. Après l'initialisation du nœud master avec kubeadm init, la commande kubeadm join a été exécutée sur le nœud worker pour l'intégrer au cluster. La commande kubectl get nodes confirme que les deux nœuds sont en état Ready.

```
azureuser@k8s-master:~$ kubectl get nodes
NAME          STATUS   ROLES           AGE     VERSION
k8s-master    Ready    control-plane   13m     v1.34.5+k3s1
k8s-worker    Ready    <none>          4m24s   v1.34.5+k3s1
azureuser@k8s-master:~$
```

Figure 23: kubectl get nodes — k8s-master (control-plane) et k8s-worker Ready, version v1.28

5. Développement et Conteneurisation de l'Application

5.1. Description de l'Application

L'application est un **gestionnaire de tâches** développé dans le cadre d'un projet DevOps complet. Il s'agit d'une application web full-stack composée d'un backend en Python/Flask exposant une API REST, et d'une interface utilisateur intégrée directement dans le serveur, accessible via un navigateur web.
L'interface adopte un thème visuel inspiré de l'esthétique des imprimeries des années 1920, avec une typographie à empattements, une palette de couleurs sépia et des encadrés à double filet, offrant une expérience utilisateur soignée et distinctive.

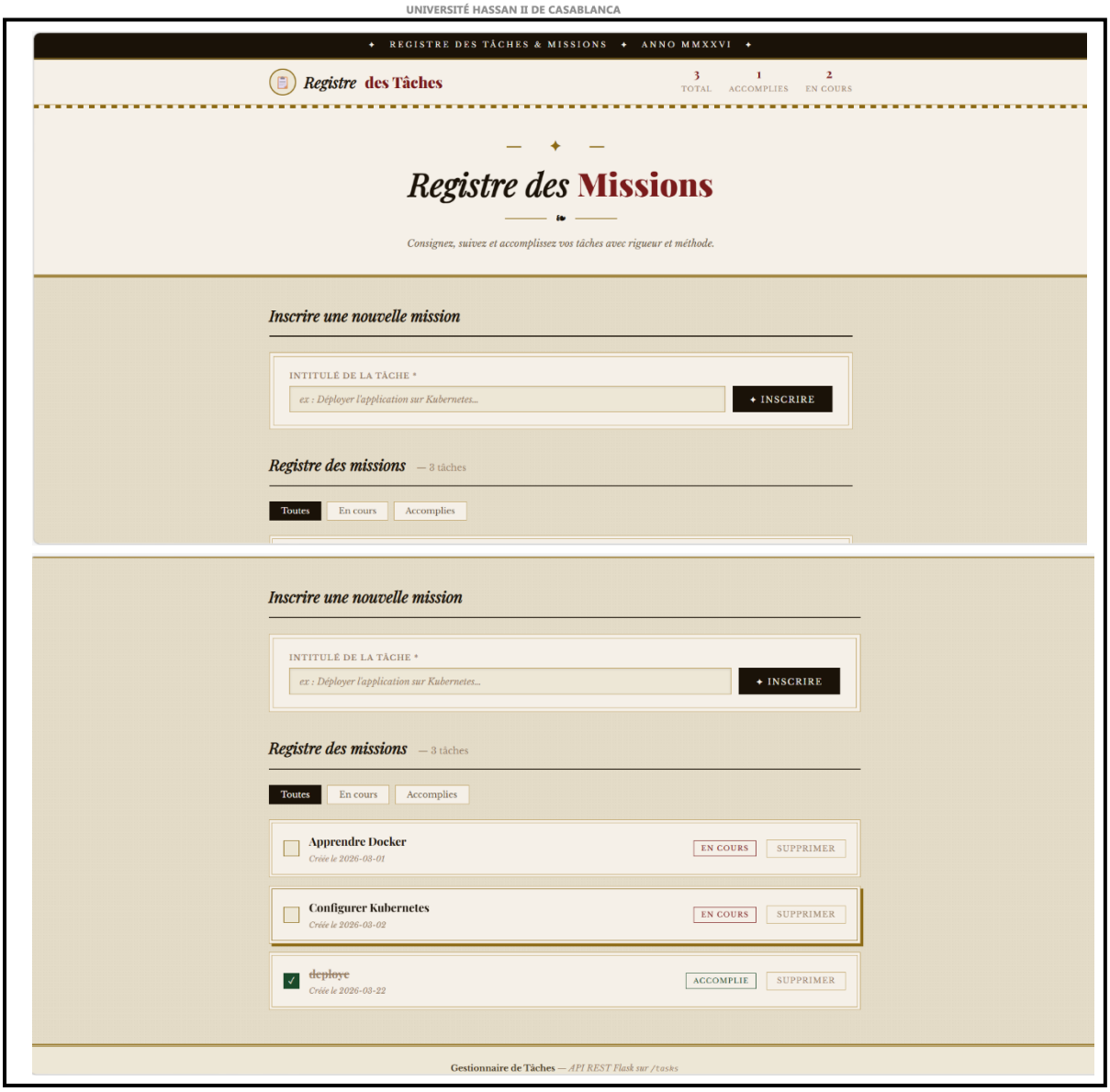
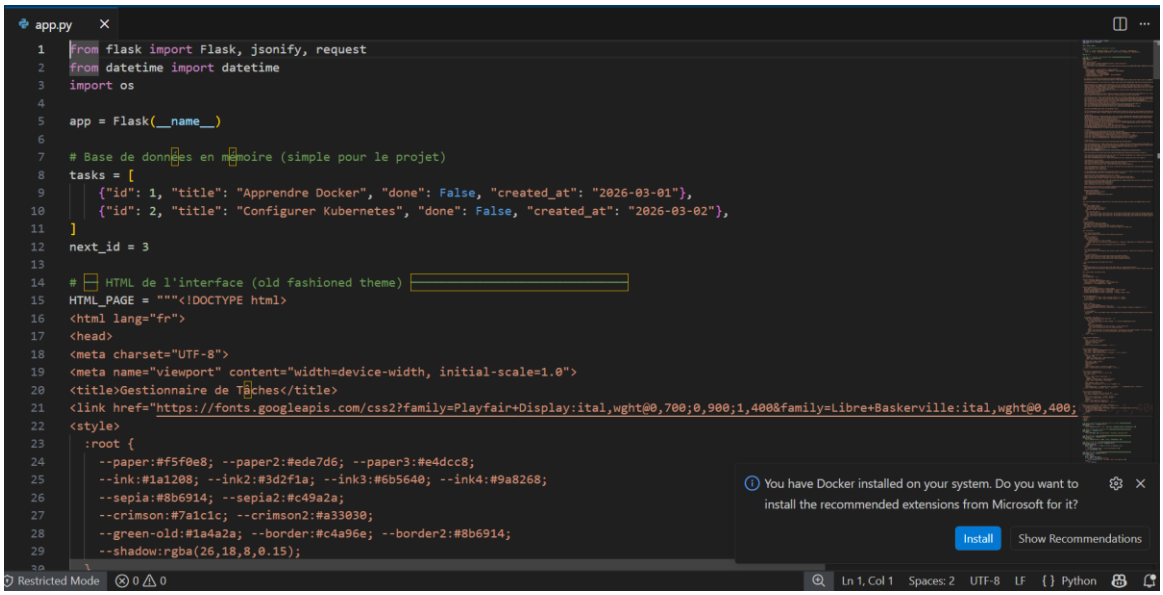


Figure 24: Interface de l'application



```python
from flask import Flask, jsonify, request
from datetime import datetime
import os

app = Flask(__name__)

# Base de données en mémoire (simple pour le projet)
tasks = [
    {"id": 1, "title": "Apprendre Docker", "done": False, "created_at": "2026-03-01"},
    {"id": 2, "title": "Configurer Kubernetes", "done": False, "created_at": "2026-03-02"},
]
next_id = 3

# ── HTML de l'interface (old fashioned theme) ──────────────
HTML_PAGE = """<!DOCTYPE html>
<html lang="fr">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Gestionnaire de Tâches</title>
<link href="https://fonts.googleapis.com/css2?family=Playfair+Display:ital,wght@0,700;0,900;1,400&family=Libre+Baskerville:ital,wght@0,400;
<style>
  :root {
    --paper:#f5f0e8; --paper2:#ede7d6; --paper3:#e4dcc8;
    --ink:#1a1208; --ink2:#3d2f1a; --ink3:#6b5640; --ink4:#9a8268;
    --sepia:#8b6914; --sepia2:#c49a2a;
    --crimson:#7a1c1c; --crimson2:#a33030;
    --green-old:#1a4a2a; --border:#c4a96e; --border2:#8b6914;
    --shadow:rgba(26,18,8,0.15);
```

Figure 25: Code source de l'application

5.2. Construction et Publication Docker

L'image a été construite avec docker build -t salama12/devops-app:1.0 ~/app/. Après résolution d'un problème de permissions (ajout au groupe docker), la construction a réussi en 12 secondes. L'image a été publiée sur DockerHub dans le dépôt public salama12/devops-app.

```
azureuser@k8s-master:~$ mkdir -p ~/k8s ~/app
azureuser@k8s-master:~$ cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Namespace
metadata:
  name: devops-app
EOF
namespace/devops-app created
azureuser@k8s-master:~$ cat <<EOF > ~/app/Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
HEALTHCHECK --interval=30s --timeout=3s \
  CMD curl -f http://localhost:5000/health || exit 1
CMD ["python", "app.py"]
EOF
azureuser@k8s-master:~$
```

```
azureuser@k8s-master:~$ docker login

USING WEB-BASED LOGIN

i Info → To sign in with credentials on the command line, use 'docker login -u <username>'


Your one-time device confirmation code is: BLMG-NFQZ
Press ENTER to open your browser or submit your device code here: https://login.docker.com/activate

Waiting for authentication in the browser…

WARNING! Your credentials are stored unencrypted in '/home/azureuser/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
azureuser@k8s-master:~$ docker build -t salama12/devops-app:1.0 ~/app/
ERROR: permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
azureuser@k8s-master:~$ sudo usermod -aG docker azureuser
azureuser@k8s-master:~$ newgrp docker
azureuser@k8s-master:~$ docker build -t salama12/devops-app:1.0 ~/app/
[+] Building 12.0s (11/11) FINISHED                                                    docker:default
 => [internal] load build definition from Dockerfile                                            0.0s
 => => transferring dockerfile: 286B                                                            0.0s
 => [internal] load metadata for docker.io/library/python:3.11-slim                             1.2s
 => [auth] library/python:pull token for registry-1.docker.io                                   0.0s
```

```
azureuser@k8s-master:~$ docker push salama12/devops-app:1.0
The push refers to repository [docker.io/salama12/devops-app]
43272e9c9d4b: Pushed
2aeefc2f798d: Pushed
206356c42440: Pushed
15eac58c3838: Pushed
206709444425: Pushed
74d466dcf4cb: Pushed
61bb3693a6db: Pushed
6fc9303757ec: Pushed
bbf418262cbe: Pushed
1.0: digest: sha256:ae1f17398c2b0ba387524e9d6bf41a6fc8b66776c8061b8aec7c157642a253f0 size: 856
azureuser@k8s-master:~$
```

Figure 26: : docker build réussi — Construction en 12s, push vers DockerHub salama12/devops-app:1.0

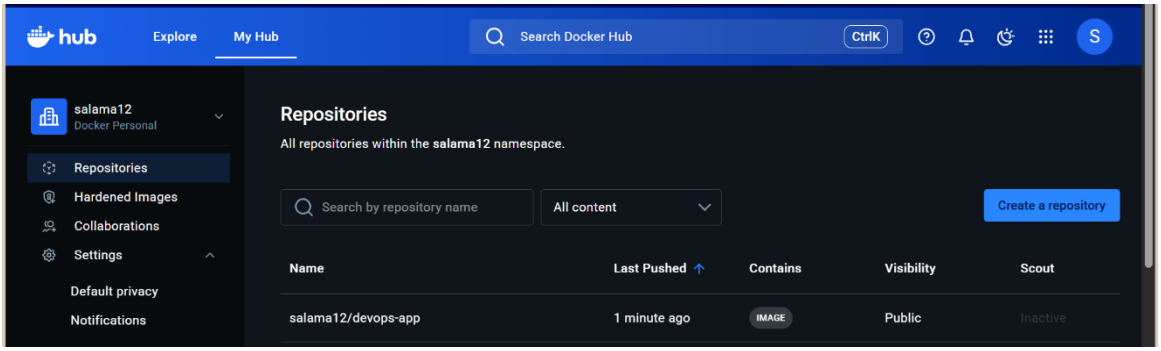


Figure 27: DockerHub — Dépôt salama12/devops-app visible publiquement, image poussée

6. Déploiement sur Kubernetes

6.1. Manifests Kubernetes

Deux fichiers YAML ont été créés : deployment.yml (Deployment avec 2 replicas, image salama12/devops-app:latest, liveness probe sur /health:5000) et service.yml (Service NodePort exposant l'application sur le port 30080 du worker).

L'utilisation d'un Deployment avec plusieurs replicas (2 pods) permet d'assurer la haute disponibilité de l'application. En cas de défaillance d'un pod, Kubernetes redémarre automatiquement un nouveau pod, garantissant ainsi la continuité de service sans interruption.

```
PS C:\Users\Salama\Desktop\devops-project> @"
>> apiVersion: apps/v1
>> kind: Deployment
>> metadata:
>>   name: devops-app
>>   namespace: devops-app
>> spec:
>>   replicas: 2
>>   selector:
>>     matchLabels:
>>       app: devops-app
>>   template:
>>     metadata:
>>       labels:
>>         app: devops-app
>>     spec:
>>       containers:
>>       - name: app
>>         image: salama12/devops-app:latest
>>         ports:
>>         - containerPort: 5000
>>         livenessProbe:
>>           httpGet:
>>             path: /health
>>             port: 5000
>> "@ | Out-File -FilePath k8s\deployment.yml -Encoding utf8
```

Figure 28: Fichier deployment.yml

```
PS C:\Users\Salama\Desktop\devops-project> @"
>> apiVersion: v1
>> kind: Service
>> metadata:
>>   name: devops-app-service
>>   namespace: devops-app
>> spec:
>>   type: NodePort
>>   selector:
>>     app: devops-app
>>   ports:
>>   - port: 80
>>     targetPort: 5000
>>     nodePort: 30080
>> "@ | Out-File -FilePath k8s\service.yml -Encoding utf8
```

Figure 29: Le fichier service.yml

6.2. Application et Vérification

Un namespace devops-app a été créé pour isoler les ressources. Les manifests ont été appliqués avec kubectl apply -f. Les pods ont démarré en état Running en moins de 3 minutes. L'endpoint /health répond correctement avec {"status":"ok","timestamp":"..."}.

```
azureuser@k8s-master:~$ kubectl get pods -n devops-app
NAME                          READY   STATUS    RESTARTS   AGE
devops-app-55ff57c744-dh6bz   1/1     Running   0          2m57s
devops-app-55ff57c744-glm7b   1/1     Running   0          2m57s
azureuser@k8s-master:~$ kubectl get svc -n devops-app
NAME                 TYPE       CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
devops-app-service   NodePort   10.43.149.21   <none>        80:30080/TCP   3m8s
azureuser@k8s-master:~$
```

```
azureuser@k8s-master:~$ kubectl get pods -n devops-app -o wide
NAME                          READY   STATUS    RESTARTS   AGE   IP          NODE         NOMINATED NODE   READINESS GATES
devops-app-55ff57c744-dh6bz   1/1     Running   0          32m   10.42.1.3   k8s-worker   <none>           <none>
devops-app-55ff57c744-glm7b   1/1     Running   0          32m   10.42.0.9   k8s-master   <none>           <none>
azureuser@k8s-master:~$ curl http://10.42.0.9:5000/health
{"status":"ok","timestamp":"2026-03-15T19:57:57.584927"}
```

Figure 30: : kubectl get pods -n devops-app — 2 pods Running, service NodePort 80:30080/TCP créé

```
azureuser@k8s-worker:~$ curl http://10.42.1.3:5000/health
{"status":"ok","timestamp":"2026-03-15T19:59:03.215033"}
azureuser@k8s-worker:~$
```

Figure 31: curl /health depuis worker — Réponse {"status":"ok"} confirmant le déploiement

7. Pipeline CI/CD avec GitHub Actions

7.1. Structure du Dépôt GitHub

Le code source est organisé dans le dépôt malikachaaban/devops-master avec la structure suivante :

- .github/workflows/deploy.yml — Pipeline CI/CD automatisé
- app/ — Application Python Flask + tests unitaires
- k8s/ — Manifests Kubernetes (deployment.yml + service.yml)
- ansible/ — Playbooks pour l'installation Docker et Kubernetes
- terraform/ — Scripts HCL pour l'infrastructure Azure
- Dockerfile — Instructions de construction de l'image

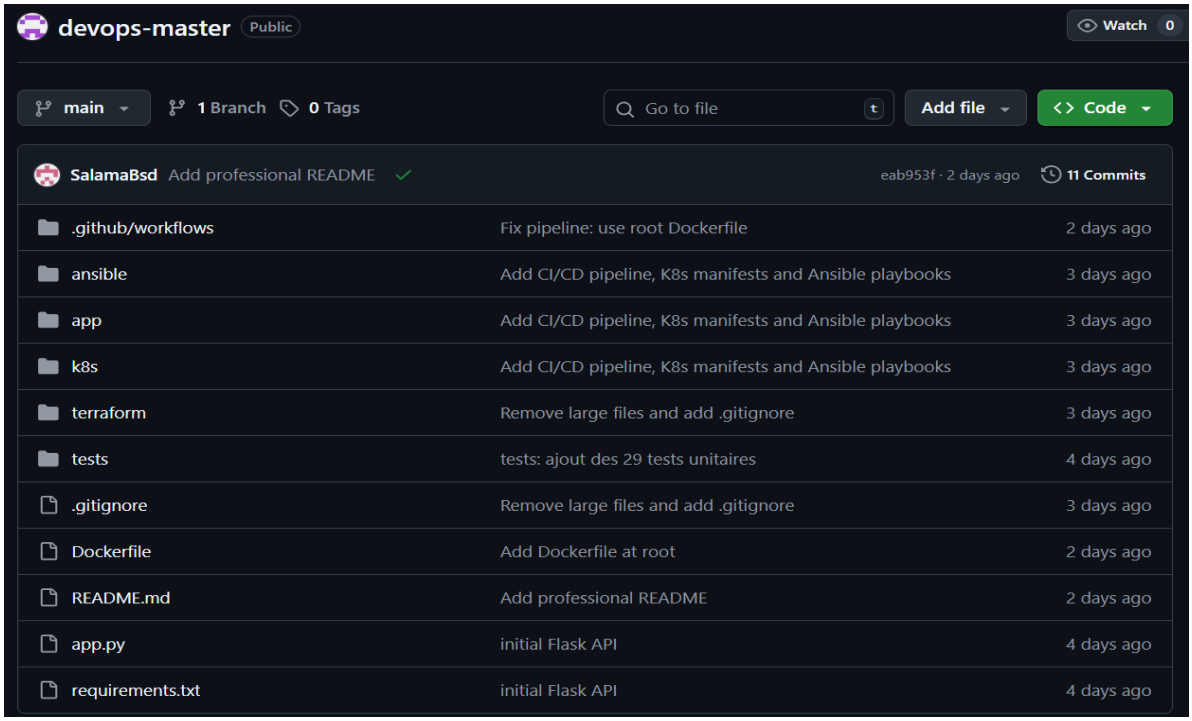


Figure 32: : Dépôt GitHub devops-master — Structure complète du projet, 9 commits

7.2. Configuration du Pipeline

Le pipeline se déclenche automatiquement à chaque push sur la branche main et exécute quatre étapes séquentielles : authentification DockerHub, construction de l'image Docker, publication sur DockerHub, déploiement Kubernetes via SSH avec kubectl rollout restart.

```python
import pytest
import app as app_module
from app import app as flask_app

# ── Fixtures ──────────────────────────────────────────────

@pytest.fixture(autouse=True)
def reset_tasks():
    """
    Remet la liste des tâches à l'état initial avant chaque test.
    Indispensable car votre app.py utilise une liste globale.
    """
    app_module.tasks.clear()
    app_module.tasks.extend([
        {"id": 1, "title": "Apprendre Docker", "done": False, "created_at": "2026-03-01"},
        {"id": 2, "title": "Configurer Kubernetes", "done": False, "created_at": "2026-03-02"},
    ])
    app_module.next_id = 3
    yield
    app_module.tasks.clear()

@pytest.fixture
def app():
    flask_app.config["TESTING"] = True
    yield flask_app
```

Figure 33: Fichier de tests unitaires test_app.py – fixtures de réinitialisation et configuration de l'application Flask en mode test

Le pipeline CI/CD inclut également une étape de tests automatisés permettant de valider le bon fonctionnement de l'application avant son déploiement. Ces tests sont exécutés automatiquement à chaque push grâce à pytest, garantissant ainsi la qualité du code et la stabilité de l'application déployée.

```
name: CI/CD Pipeline
>>
>> on:
>>   push:
>>     branches: [main]
>>
>> jobs:
>>   build-and-deploy:
>>     runs-on: ubuntu-latest
>>
>>     steps:
>>       - name: Checkout code
>>         uses: actions/checkout@v3
>>
>>       - name: Login to DockerHub
>>         run: echo "`${{ secrets.DOCKERHUB_PASSWORD }}" | docker
login -u "`${{ secrets.DOCKERHUB_USERNAME }}" --password-stdin
>>
>>       - name: Install dependencies
>>         run: pip install flask pytest
>>
>>       - name: Run tests
>>         run: cd app && pytest tests/ -v
>>
>>       - name: Build Docker image
>>         run: docker build -t `${{ secrets.DOCKERHUB_USERNAME
}}/devops-app:latest .
>>
>>       - name: Push to DockerHub
```

```
>>          run: docker push `${{ secrets.DOCKERHUB_USERNAME
}}/devops-app:latest
>>
>>        - name: Deploy to Kubernetes via SSH
>>          uses: appleboy/ssh-action@v0.1.10
>>          with:
>>            host: `${{ secrets.MASTER_IP }}
>>            username: azureuser
>>            key: `${{ secrets.SSH_PRIVATE_KEY }}
>>            script: |
>>              kubectl rollout restart deployment/devops-app -n
devops-app
>>              kubectl rollout status deployment/devops-app -n
devops-app
```

```
Salama@DESKTOP-03KVC3V MINGW64 ~/Desktop/devops-project (master)
$ git commit -m "Add CI/CD pipeline, K8s manifests and Ansible playbooks"
[master (root-commit) 9efbc1f] Add CI/CD pipeline, K8s manifests and Ansible playbooks
 26 files changed, 1973 insertions(+)
 create mode 100644 .github/workflows/deploy.yml
 create mode 100644 ansible/install_docker.yml
 create mode 100644 ansible/install_k8s.yml
 create mode 100644 ansible/inventory.ini
 create mode 100644 app/README.md
 create mode 100644 app/app.py
 create mode 100644 app/requirements.txt
 create mode 100644 app/tests/__init__.py
 create mode 100644 app/tests/test_app.py
 create mode 100644 k8s/deployment.yml
 create mode 100644 k8s/service.yml
 create mode 100644 terraform/.terraform.lock.hcl
 create mode 100644 terraform/.terraform/providers/registry.terraform.io/hashicorp/azurerm/3.117.1/windows_amd64/LICENSE.t
xt
 create mode 100644 terraform/.terraform/providers/registry.terraform.io/hashicorp/azurerm/3.117.1/windows_amd64/terraform
-provider-azurerm_v3.117.1_x5.exe
 create mode 100644 terraform/.terraform/providers/registry.terraform.io/hashicorp/local/2.7.0/windows_amd64/LICENSE.txt
 create mode 100644 terraform/.terraform/providers/registry.terraform.io/hashicorp/local/2.7.0/windows_amd64/terraform-pro
vider-local_v2.7.0_x5.exe
 create mode 100644 terraform/.terraform/providers/registry.terraform.io/hashicorp/tls/4.2.1/windows_amd64/LICENSE.txt
 create mode 100644 terraform/.terraform/providers/registry.terraform.io/hashicorp/tls/4.2.1/windows_amd64/terraform-provi
der-tls_v4.2.1_x5.exe
 create mode 100644 terraform/devops-key.pem
 create mode 100644 terraform/main.tf
 create mode 100644 terraform/outputs.tf
 create mode 100644 terraform/provider.tf
 create mode 100644 terraform/terraform.tfstate
 create mode 100644 terraform/terraform.tfstate.backup
 create mode 100644 terraform/terraform.tfvars
 create mode 100644 terraform/variables.tf
```

Figure 34: Initialisation du dépôt et push du projet vers GitHub (branche main)

```
Salama@DESKTOP-03KVC3V MINGW64 ~/Desktop/devops-project (main)
$ git commit --allow-empty -m "Trigger CI/CD with secrets"
[main 5a050ef] Trigger CI/CD with secrets

Salama@DESKTOP-03KVC3V MINGW64 ~/Desktop/devops-project (main)
$ git push origin main
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 209 bytes | 209.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/malikachaaban/devops-master.git
   d024ddf..5a050ef  main -> main
```

Figure 35: Configuration du pipeline CI/CD GitHub Actions pour le build, les tests et le déploiement automatique

Les tests sont exécutés automatiquement à chaque push afin de détecter les erreurs avant le déploiement et garantir la qualité du code.

7.3. Secrets et Résultats

Quatre secrets GitHub ont été configurés pour sécuriser les informations sensibles : DOCKERHUB_USERNAME, DOCKERHUB_PASSWORD, MASTER_IP (68.221.161.55) et SSH_PRIVATE_KEY. Le pipeline #4 s'est exécuté avec succès en 27 secondes, validant l'ensemble de la chaîne DevOps.

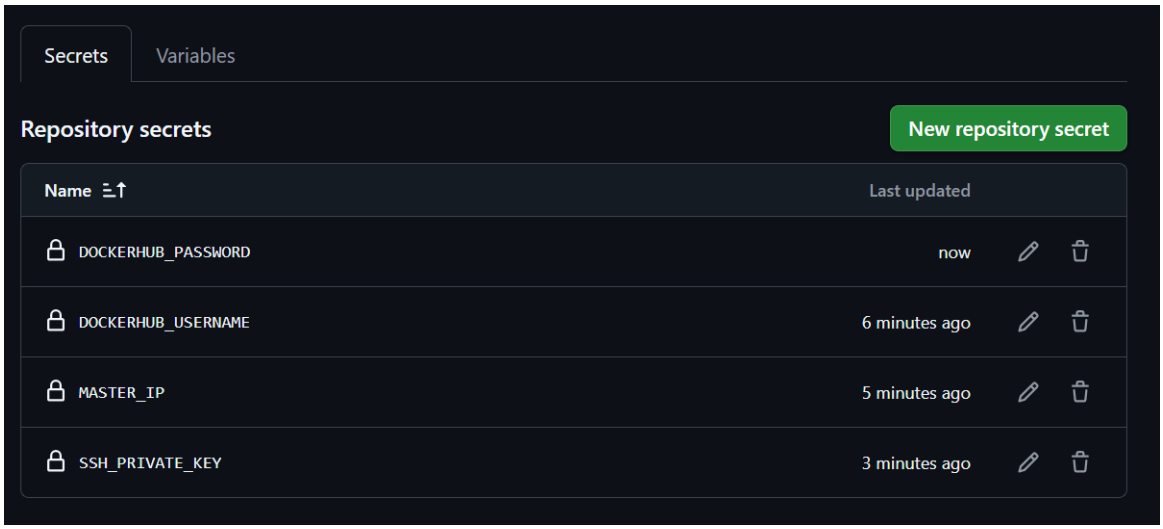


Figure 36: Secrets GitHub configurés — 4 secrets chiffrés pour sécuriser le déploiement automatique

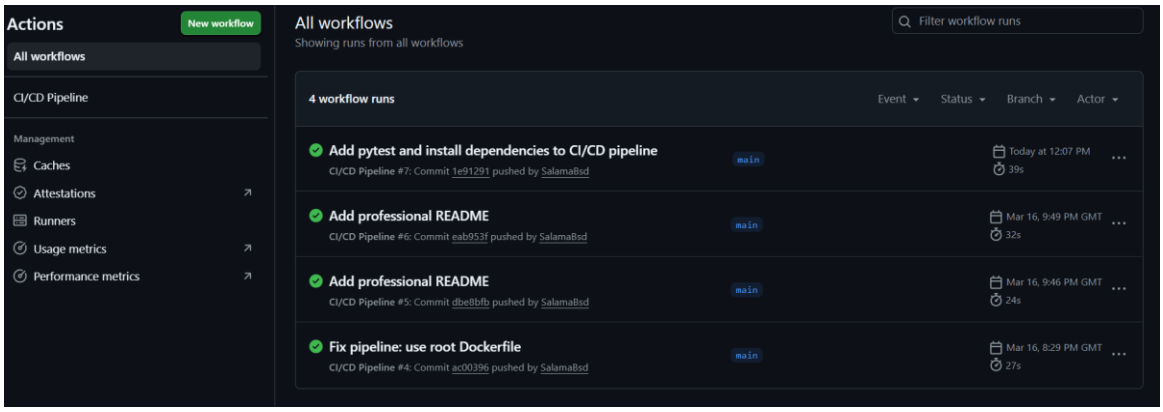


Figure 37: GitHub Actions — Pipeline CI/CD #4 vert en 27 secondes, toutes les étapes réussies

8. Automatisation avec Ansible

8.1. Inventaire

Le fichier inventory.ini définit les deux groupes d'hôtes. Le groupe [master] pointe vers 68.221.161.55 et le groupe [worker] vers 20.251.155.11. Les deux utilisent l'utilisateur azureuser avec la clé SSH privée devops-key.pem.

```
PS C:\Users\Salama> cat C:\Users\Salama\Desktop\devops-project\ansible\inventory.ini
[master]
68.221.161.55 ansible_user=azureuser ansible_ssh_private_key_file=~/.ssh/devops-key.pem

[worker]
20.251.155.11 ansible_user=azureuser ansible_ssh_private_key_file=~/.ssh/devops-key.pem
```

Figure 38: Le fichier inventory.ini des groupes hôtes

8.2. Playbook Docker (install_docker.yml)

Ce playbook cible tous les hôtes et utilise le module apt pour installer docker.io, puis le module service pour activer et démarrer Docker au démarrage du système. Il constitue la documentation exécutable de la configuration des serveurs.

```
PS C:\Users\Salama> cat C:\Users\Salama\Desktop\devops-project\ansible\install_docker.yml
- hosts: all
  become: yes
  tasks:
    - name: Install Docker
      apt:
        name: docker.io
        state: present
        update_cache: yes
    - name: Start Docker
      service:
        name: docker
        state: started
        enabled: yes
```

Figure 39: Le fichier install_docker.yml

8.3. Playbook Kubernetes (install_k8s.yml)

Ce playbook automatise l'installation de kubeadm, kubelet et kubectl sur les deux nœuds via le module shell, en utilisant le dépôt officiel kubernetes.io. La commande apt-mark hold est incluse pour bloquer les mises à jour automatiques.

```
PS C:\Users\Salama> cat C:\Users\Salama\Desktop\devops-project\ansible\install_k8s.yml
- hosts: all
  become: yes
  tasks:
    - name: Install kubeadm kubelet kubectl
      shell: |
        apt-get update
        apt-get install -y kubelet kubeadm kubectl
        apt-mark hold kubelet kubeadm kubectl
```

Figure 40: Le fichier install_k8s.yml

L'utilisation d'Ansible permet d'automatiser entièrement la configuration des machines, évitant toute intervention manuelle et garantissant la reproductibilité de l'environnement. Cette approche s'inscrit pleinement dans les principes DevOps d'automatisation et d'Infrastructure as Code.

8.4. Reproduction de l'Infrastructure

7. Accéder à l'application : http://<IP_WORKER>:30080

6. Déployer l'application : kubectl apply -f k8s/

5. Rattacher le worker : sudo kubeadm join <MASTER_IP>:6443 --token <TOKEN>

4. Initialiser le cluster (master) : sudo kubeadm init --pod-network-cidr=192.168.0.0/16

3. Configurer les machines avec Ansible : ansible-playbook -i ansible/inventory.ini ansible/install_docker.yml && ansible/install_k8s.yml

2. Déployer l'infrastructure : cd terraform && terraform init && terraform apply

1. Cloner le dépôt : git clone https://github.com/malikachaaban/devops-master.git

Afin de garantir la reproductibilité du projet, les étapes suivantes permettent de déployer l'infrastructure complète à partir de zéro :

```
1. git clone https://github.com/malikachaaban/devops-master.git
2. cd terraform && terraform init && terraform apply
3. ansible-playbook -i ansible/inventory.ini
ansible/install_docker.yml
4. ansible-playbook -i ansible/inventory.ini
ansible/install_k8s.yml
5. sudo kubeadm init --pod-network-cidr=192.168.0.0/16
6. sudo kubeadm join <MASTER_IP>:6443 --token <TOKEN> --discovery-
token-ca-cert-hash <HASH>
7. kubectl apply -f k8s/ && curl http://<IP_WORKER>:30080/health
```

9. Monitoring avec Grafana

9.1. Installation de Grafana avec Helm

Dans le cadre des objectifs optionnels du projet, un système de monitoring a été mis en place en utilisant Grafana, déployé via Helm sur le cluster Kubernetes dans un namespace dédié "monitoring". Helm est un gestionnaire de packages pour Kubernetes qui simplifie le déploiement d'applications complexes. L'installation a utilisé le chart officiel grafana/grafana avec des ressources limitées adaptées à la VM Standard_B2s.

Les étapes d'installation ont été les suivantes :

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
helm repo add grafana https://grafana.github.io/helm-charts
helm repo update
helm install grafana grafana/grafana \
  --namespace monitoring --create-namespace \
  --set resources.requests.memory=64Mi \
  --set resources.limits.memory=128Mi \
  --set persistence.enabled=false
```

```
azureuser@k8s-master:~$ curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 11929  100 11929    0     0   144k      0 --:--:-- --:--:-- --:--:--  145k
Downloading https://get.helm.sh/helm-v3.20.1-linux-amd64.tar.gz
Verifying checksum... Done.
Preparing to install helm into /usr/local/bin
helm installed into /usr/local/bin/helm
azureuser@k8s-master:~$ helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
"prometheus-community" has been added to your repositories
azureuser@k8s-master:~$ helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "prometheus-community" chart repository
Update Complete. ⎈Happy Helming!⎈
```

```
azureuser@k8s-master:~$ helm install grafana grafana/grafana \
  --namespace monitoring \
  --create-namespace \
  --set resources.requests.memory=64Mi \
  --set resources.limits.memory=128Mi \
  --set resources.requests.cpu=50m \
  --set persistence.enabled=false
```

Figure 41: Etapes d'installation

L'installation s'est terminée avec succès (STATUS: deployed). Le pod Grafana a démarré correctement en état Running dans le namespace monitoring sur le nœud worker.

9.2. Accès au Dashboard Grafana

Grafana a été exposé via un Service de type NodePort sur le port 30300, accessible depuis l'IP publique du nœud worker. Le mot de passe administrateur a été récupéré depuis le secret Kubernetes créé automatiquement lors de l'installation.

```
kubectl patch svc grafana -n monitoring \
  -p
'{"spec":{"type":"NodePort","ports":[{"port":80,"nodePort":30300,"
targetPort":3000}]}}'
```

```
azureuser@k8s-master:~$ kubectl patch svc grafana -n monitoring \
  -p '{"spec":{"type":"NodePort","ports":[{"port":80,"nodePort":30300,"targetPort":3000}]}}'
service/grafana patched (no change)
azureuser@k8s-master:~$
```

Figure 42: Exposition du service Grafana via NodePort sur Kubernetes

Les informations d'accès au dashboard Grafana sont les suivantes : URL d'accès : `http://20.251.155.11:30300` | Identifiant : admin | Mot de passe récupéré via kubectl depuis le secret Kubernetes.

9.3. Vérification du Déploiement

La commande kubectl get pods -n monitoring confirme que le pod Grafana est en état Running sur le nœud worker (IP 10.42.1.13). Le service NodePort est correctement configuré avec le mapping 80:30300/TCP, permettant l'accès externe au dashboard de monitoring.

```
NAME                        READY   STATUS    RESTARTS   AGE   IP
NODE
grafana-7bd5db94ff-x7prb    1/1     Running   0          79m
10.42.1.13   k8s-worker
```

```
azureuser@k8s-master:~$ kubectl get pods -n monitoring
NAME                        READY   STATUS    RESTARTS   AGE
grafana-7bd5db94ff-x7prb    1/1     Running   0          107m
azureuser@k8s-master:~$
```

Figure 43: Vérification de l'état du pod Grafana dans le namespace monitoring

```
azureuser@k8s-master:~$ kubectl get pods -n monitoring -o wide
NAME                        READY   STATUS    RESTARTS   AGE   IP          NODE        NOMINATED NODE   READINESS GATES
grafana-7bd5db94ff-x7prb    1/1     Running   0          79m   10.42.1.13  k8s-worker  <none>           <none>
```

Figure 44: Affichage détaillé du pod Grafana avec son adresse IP et son nœud d'exécution

10. Conclusion

10.1. Récapitulatif

Ce projet a permis de mettre en place une infrastructure DevOps complète couvrant l'ensemble du cycle de vie d'une application. Toutes les composantes ont été réalisées avec succès :

- Infrastructure Azure — 2 VMs Ubuntu déployées et opérationnelles
- Terraform — Configuration complète, terraform validate Success
- Kubernetes — Cluster opérationnel, master + worker en état Ready
- Docker — Image construite et publiée sur DockerHub
- Application Flask — API REST avec 6 endpoints et health check
- Déploiement Kubernetes — 2 pods Running, service NodePort accessible
- Pipeline CI/CD — GitHub Actions vert en 27 secondes
- Ansible — Playbooks Docker et Kubernetes documentés

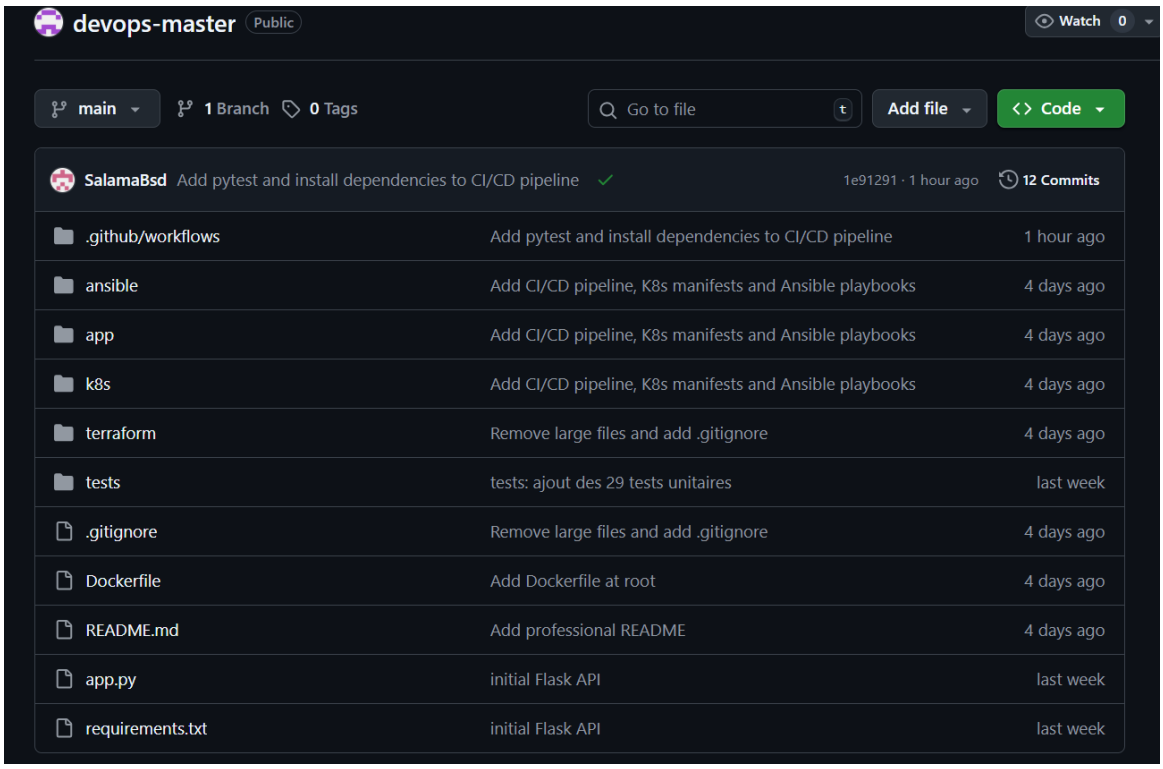


Figure 45: Structure du projet.

10.2. Compétences Acquises

La réalisation de ce projet a permis d'acquérir des compétences pratiques en Infrastructure as Code (Terraform), conteneurisation (Docker), orchestration (Kubernetes), automatisation CI/CD (GitHub Actions) et configuration de serveurs (Ansible). Les difficultés rencontrées — gestion des fichiers Git volumineux, permissions Docker, débogage du pipeline — ont été des opportunités d'apprentissage précieuses. Ce pipeline illustre l'approche DevOps en automatisant l'intégration, les tests et le déploiement continu de l'application.

11. Références

- Documentation Microsoft Azure : https://docs.microsoft.com/azure
- Documentation Terraform Azure Provider : https://registry.terraform.io/providers/hashicorp/azurerm
- Documentation officielle Kubernetes : https://kubernetes.io/docs
- Documentation Docker : https://docs.docker.com
- Documentation GitHub Actions : https://docs.github.com/actions
- Documentation Ansible : https://docs.ansible.com
- Documentation Python Flask : https://flask.palletsprojects.com
- Documentation k3s (Rancher) : https://docs.k3s.io
- DockerHub Repository : https://hub.docker.com/r/salama12/devops-app
- Dépôt GitHub du projet : https://github.com/malikachaaban/devops-master